

# Chapter 1

## The Travelling Salesperson Problem

Nature Inspired Optimisation for Delivery Problems (2022)

Slides based on the textbook

2022

# Chapter Overview

- 1 1.1 A Brief History of the TSP
- 2 1.2 Problem Description
- 3 1.3 Techniques for Solving the TSP
- 4 1.4 Software Engineering Considerations
- 5 1.5 A TSP Case Study: Santa Claus
- 6 1.6 Conclusions

## 1.1 A Brief History of the TSP I

The **Travelling Salesperson Problem** (TSP) is one of the most famous and extensively studied combinatorial optimisation problems in mathematics and computer science. Although it sounds simple — visit a set of cities, return home, minimise the total distance travelled — it has proved extraordinarily difficult to solve in general, and that difficulty has motivated decades of algorithmic research.

**Origins.** The problem was first described in mathematical terms by the Irish mathematician **W. R. Hamilton** and the British mathematician **Thomas Kirkman** in the 1800s. Hamilton posed the “Icosian Game” (1857), which asked for a Hamiltonian circuit — a closed path that visits every vertex of a graph exactly once — on the edges of an icosahedron.

**The name and modern formulation.** The term “Travelling Salesman Problem” was introduced by **Merrill Flood** and **Melvin Dresher** at the RAND Corporation in the 1940s. **Karl Menger** had already noted in Vienna in the 1930s that naïve exhaustive search would be computationally infeasible even for moderate-sized instances.

**Key milestone.** The first computerised solution appeared in 1958: **Bocks (1958)** described solving the TSP using an algorithm implemented on an **IBM 650** computer, marking the beginning of the computational era for combinatorial optimisation.

## 1.1 A Brief History of the TSP II

**Why the TSP remains important.** The TSP is important not merely for routing salespeople. Because it captures the essence of NP-hard combinatorial search, virtually every delivery, logistics, or routing problem in the modern world inherits the TSP's difficulty. Progress on the TSP — exact methods, heuristics, metaheuristics — directly translates into better algorithms for vehicle routing, circuit board drilling, genome sequencing, and telescope scheduling.

## 1.2 Problem Description I

### Informal problem statement

A salesperson must visit a number of cities, each city visited *exactly once*, with the salesperson ending back at the starting city. The aim is to find the *shortest route* that permits all visits.

The core challenge is that we must search over all possible orderings (permutations) of the cities in order to find the best one. Such orderings are also called **Hamiltonian circuits**, named after the mathematician W. R. Hamilton mentioned in the previous section.

## 1.2 Problem Description II

### Formal elements of the TSP.

- **Nodes (cities):** A set  $V = \{v_1, v_2, \dots, v_n\}$  of  $n$  locations that must all be visited. Here  $n$  (Roman letter “n”) denotes the total number of cities.
- **Edges:** A set  $E$  of connections between pairs of cities. In the *complete* TSP every city is directly reachable from every other city, so  $E$  contains all  $n(n - 1)/2$  possible pairs.
- **Distance matrix:** A two-dimensional table  $D$  where the entry  $d(i, j)$  gives the distance (or cost) of travelling directly from city  $i$  to city  $j$ . When distances are symmetric —  $d(i, j) = d(j, i)$  for all pairs — the problem is called a *symmetric* TSP.

## 1.2 Problem Description III

### Objective function

Given a tour  $\pi = (\pi_1, \pi_2, \dots, \pi_n)$  — a permutation ( $\pi$ , pronounced “pi”) of the  $n$  cities — the total tour length is:

$$L(\pi) = \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) + d(\pi_n, \pi_1)$$

### Notation explained

- $\pi$  (pi): a particular ordering of all cities, e.g.  $\pi = (A, C, B, D, E)$  means “visit A first, then C, then B, ...”.
- $\pi_i$ : the city at position  $i$  in the tour.  $\pi_1$  is the first city,  $\pi_n$  is the last city before returning home.
- $d(\pi_i, \pi_{i+1})$ : the distance between the  $i$ -th and  $(i + 1)$ -th city in the tour.
- $\sum_{i=1}^{n-1}$  (Sigma, meaning “sum over  $i$  from 1 to  $n - 1$ ”): we add up all the leg distances as we traverse the tour, then add the final return leg  $d(\pi_n, \pi_1)$ .

## 1.2 Problem Description IV

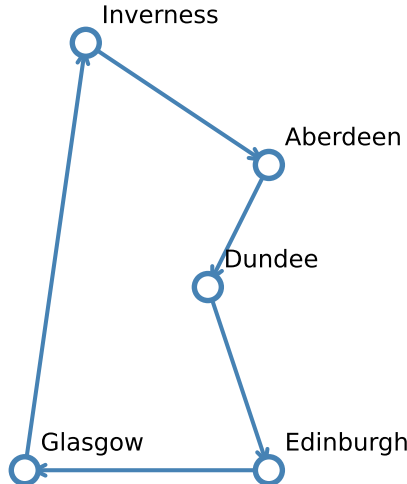
### How to read this formula

The formula simply says: add up the distances of all the segments of the journey. If you double the number of stops (roughly) and each leg has a similar length, the total distance roughly doubles. The goal is to find the ordering  $\pi$  that makes  $L(\pi)$  as small as possible.



## 1.2 Problem Description V

A valid 5-city TSP tour  
(Hamiltonian Circuit)



## 1.3 Techniques for Solving the TSP — Overview I

Because the TSP is NP-hard, practitioners use a spectrum of methods that trade *solution quality* for *computation time*:

### Taxonomy of TSP solvers

**Exact methods** Guarantee the optimal solution but may require exponential time. Practical only for small instances or with very sophisticated bounding. Examples: **exhaustive search**, **Branch and Bound**, **Concorde** (LP + cutting planes).

**Construction heuristics** Build a tour from scratch using a greedy rule. Fast, but the result may be far from optimal. Example: **Nearest Neighbour**.

**Improvement heuristics** Start with any tour and repeatedly modify it to get a shorter one. Examples: **2-opt**, **3-opt**, **Lin-Kernighan**.

**Metaheuristics** Higher-level strategies that guide local search to escape local optima. Examples: **Evolutionary Algorithms**, **Simulated Annealing**, **Ant Colony Optimisation**.

The remainder of Section 1.3 focuses on methods implemented in this chapter's case study: Exhaustive Search, Nearest Neighbour, and 2-opt.

# Exhaustive Search I

**What problem it solves.** Exhaustive search (also called *brute-force search*) enumerates every possible tour, measures its length, and keeps track of the shortest one found. It is the only method that *guarantees* the globally optimal solution — because it checks all possibilities.

**Why it eventually fails.** As we saw, the number of distinct tours is  $(n - 1)!/2$ . For 13 cities this is about 3 billion, and for 20 cities it is about  $6 \times 10^{16}$ . Even at a billion tours per second, 20 cities would take more than 60 million years. Therefore exhaustive search is only practical for very small instances (roughly  $n \leq 12$ ).

# Exhaustive Search II

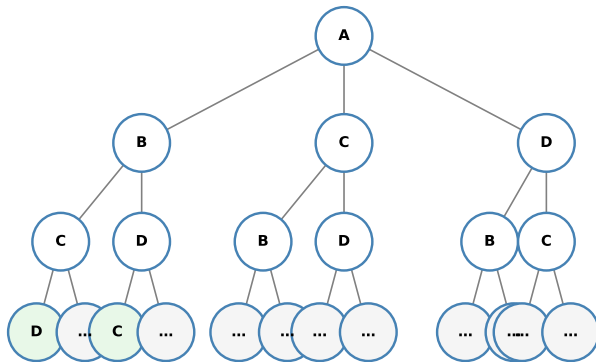
## Algorithm 1: Exhaustive Search (pseudocode)

```
1:  $notVisited \leftarrow \{1, \dots, n\}$ 
2:  $solution \leftarrow \emptyset$ 
3: for each permutation  $\pi$  of  $notVisited$  do
4:     if  $dist(\pi) < dist(solution)$  then
5:          $solution \leftarrow \pi$ 
6: return  $solution$ 
```

In practice, the permutations are generated *recursively* (see the Java listing in the case study): the `search(visits, l, r)` function swaps elements at positions `l` through `r` to enumerate all orderings without storing them all in memory at once.

# Exhaustive Search III

Partial search tree for 4-city exhaustive search (starting from A)



*Each path root→leaf is a complete tour. With  $n$  cities there are  $(n-1)!/2$  distinct tours.*

# Exhaustive Search IV

**Figure:** A partial search tree for exhaustive TSP search on 4 cities starting from city A. Each path from the root to a leaf represents one complete tour. With 4 cities there are  $(4 - 1)!/2 = 3$  distinct tours; with  $n = 15$  there are already  $\approx 43$  billion.

# The Nearest Neighbour Heuristic I

**What problem it solves.** The Nearest Neighbour (NN) heuristic is a *construction heuristic* — it builds a complete tour from scratch, one city at a time. At each step it simply visits the *closest unvisited city*. This greedy strategy is intuitive and very fast, running in  $O(n^2)$  time (quadratic in the number of cities).

**Intuition: why it works reasonably well.** By always going to the nearest unvisited city, the salesperson avoids very long “wasted” legs. However, because no look-ahead is performed, the algorithm can paint itself into a corner at the end — the last few cities added to the tour may require long detours, making the final tour sub-optimal. Figure 1.2 of the book illustrates exactly this phenomenon on a 7-city example.

# The Nearest Neighbour Heuristic II

## Algorithm 2: TSP — Nearest Neighbour

```
1:  $notVisited \leftarrow \{1, \dots, n\}$ 
2:  $solution \leftarrow \emptyset$ 
3:  $current \leftarrow \text{removeRand}(notVisited)$  // pick a random starting city
4:  $solution \leftarrow solution \cup \{current\}$ 
5: while  $notVisited \neq \emptyset$  do
6:      $best \leftarrow \infty$ 
7:     for  $possible$  in  $notVisited$  do
8:         if  $\text{dist}(current, possible) < best$  then
9:              $next \leftarrow possible$ 
10:     $notVisited \leftarrow notVisited - \{next\}$ 
11:     $solution \leftarrow solution \cup \{next\}$ 
12:     $current \leftarrow next$ 
13: return  $solution$ 
```



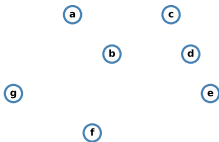
# The Nearest Neighbour Heuristic III

**Line-by-line explanation.** Line 3 picks a random starting city (making the algorithm *stochastic*). The main loop (lines 5–12) runs until all cities have been added. Lines 7–9 scan every unvisited city to find the one closest to the current position. Lines 10–12 mark it as visited, add it to the tour, and move there.

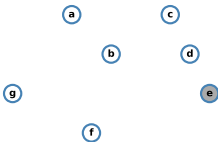
# The Nearest Neighbour Heuristic IV

## Nearest Neighbour Heuristic — Step by Step

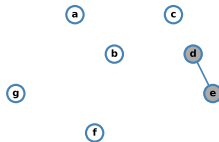
(a) Problem: 7 cities



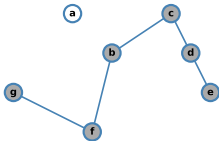
(b) Start at E (selected at random)



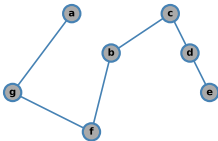
(c) Nearest to E is D



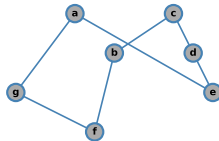
(d) Nearest unvisited: C, then B, F, G



(e) Last unvisited: A added



(f) Complete NN tour (sub-optimal)

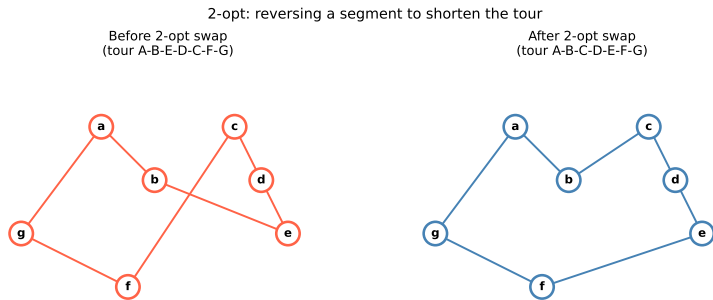


# The 2-opt Heuristic I

**What problem it solves.** The 2-opt heuristic (Croes 1958) is a *local search* improvement algorithm. It starts from an existing tour (e.g., one produced by NN) and repeatedly *improves* it. At each step it tries removing two edges from the tour and reconnecting the two resulting segments in the only other possible way. If the reconnection produces a shorter tour, it is accepted. The process repeats until no 2-edge swap can improve the tour further.

**Intuition: why it works.** An inefficient tour often contains *crossing edges* — two edges that intersect in the plane. Whenever two edges cross, swapping them (reversing the segment between the two crossing points) always yields a shorter tour. The 2-opt algorithm systematically finds and fixes these crossings.

# The 2-opt Heuristic II



**Figure:** A 7-city example of the 2-opt swap. *Left:* the tour A-B-E-D-C-F-G contains two crossing edges. *Right:* after reversing the sub-segment between the crossing points (reversing E-D-C to C-D-E), the edges no longer cross and the total tour length decreases.

# The 2-opt Heuristic III

**The 2-opt swap operation.** Given positions  $i$  and  $j$  in the tour (with  $i < j$ ), the *2-opt swap* reverses the sub-sequence between positions  $i$  and  $j$ .

## Worked example: 2-opt swap on a 10-city tour

Original tour: A B C D E F G H I J

We select positions  $i = 4$  and  $j = 8$  (0-indexed), highlighted: A B C D E F G H I J

Applying the swap reverses the bold segment: A B C H G F E D I J

The new tour is A B C H G F E D I J. If the total distance of this new tour is less than the original, we accept it; otherwise we discard it and try the next pair  $(i, j)$ .

The complete 2-opt algorithm (Algorithm 5 in the book) tries every pair of positions  $(i, j)$  with  $0 \leq i < j < n$ , which gives  $\binom{n}{2}$  possible swaps. If any swap improves the tour, it is adopted and the scan restarts. The algorithm terminates when a full scan produces no improvement.

# The 2-opt Heuristic IV

## Algorithm 4: 2-opt swap procedure

**Procedure** `swap(tour, i, j):`

```
1:  for  $x = 0; x < i; x++$  do
2:       $result[x] \leftarrow tour[x]$     // copy first segment unchanged
3:  for  $x = j; x \geq i; x--$  do
4:       $result[x] \leftarrow tour[x]$     // copy middle segment in reverse
5:  for  $x = j+1; x < tour.length; x++$  do
6:       $result[x] \leftarrow tour[x]$     // copy trailing segment unchanged
7:  return result
```

# The 2-opt Heuristic V

## Algorithm 5: Full 2-opt

```
1:  solution  $\leftarrow$  randomSolution()
2:  finished  $\leftarrow$  false
3:  do
4:    improved  $\leftarrow$  false
5:    for  $x = 0; x < n; x++$  do
6:      for  $y = x+1; y < n; y++$  do
7:        solution'  $\leftarrow$  swap(solution,  $x, y$ )
8:        if dist(solution') < dist(solution) then
9:          solution  $\leftarrow$  solution'; improved  $\leftarrow$  true; break
10: while improved
11: return solution
```

**Line-by-line explanation of Algorithm 5.** Line 1 starts from a random (or NN-generated) tour. The outer **do-while** loop (lines 3–10) keeps running as long as at least one improvement was found in the

## The 2-opt Heuristic VI

previous scan. Lines 5–6 iterate over all pairs  $(x, y)$  of positions. Line 7 creates the swapped tour. Line 8 checks whether the swap reduces the total distance; if so, line 9 adopts the new tour, sets the improved flag, and breaks out of the inner loops to restart the scan from the beginning. When a complete scan (all  $\binom{n}{2}$  pairs) finds no improvement, the algorithm has reached a *local optimum* and terminates.



# The 2-opt Heuristic VII

## The iterative improvement pattern (Algorithm 3)

```
1: solution  $\leftarrow$  randomSolution()
2: while  $\neg$ done do
3:   solution'  $\leftarrow$  randomChange(solution)
4:   if quality(solution') < quality(solution) then
5:     solution  $\leftarrow$  solution'
6: return solution
```

This is a fundamental design pattern that appears repeatedly throughout the book: generate a random change, keep it only if it improves the objective. The 2-opt algorithm is a specific instance of this pattern where the “random change” is a specific 2-edge swap and the improvement criterion is a shorter total tour distance.

# The 2-opt Heuristic VIII

## Key takeaway: 2-opt

2-opt runs in  $O(n^2)$  per pass and typically improves NN solutions by 10–20%. It is guaranteed to find a 2-opt *local optimum* but not necessarily the global optimum. Combining NN initialisation with 2-opt refinement — the “Hybrid” approach — gives the best results among the methods studied in this chapter.

# Other TSP Techniques (Overview) I

**3-opt and Lin-Kernighan.** The 3-opt heuristic generalises 2-opt by removing *three* edges and reconnecting the tour in the best possible way. The **Lin-Kernighan** (LK) heuristic (1973) is a variable-depth local search that removes  $k$  edges in sequence, choosing the most promising  $k$  at each step. LK and its extensions (LKH) currently produce the best heuristic solutions known for large TSP instances.

**Exact methods: Branch and Bound.** Branch and Bound (B&B) is a tree-search algorithm that systematically explores partial tours and *prunes* any branch whose best possible extension cannot beat the current best solution. The key operation is computing a *lower bound* on the cost of completing a partial tour; the 1-tree relaxation (a minimum spanning tree extended to include two edges at one vertex) provides a classic tight lower bound.

**Concorde.** Concorde (Applegate et al., 2006) is the state-of-the-art exact TSP solver. It combines linear programming relaxations with cutting-plane algorithms (adding inequalities that cut off non-tour fractional solutions) and branch-and-cut. Concorde has solved instances with tens of thousands of cities to proven optimality.

**Evolutionary algorithms.** Evolutionary algorithms (EAs) maintain a *population* of tours and apply *crossover* (combining sub-tours from two parents) and *mutation* (small random changes) to generate new

## Other TSP Techniques (Overview) II

tours. The best-quality individuals survive to the next generation. EAs are covered in later chapters of this book as the primary class of nature-inspired optimisation methods for delivery problems.

### Key takeaway: choosing a method

For small instances ( $n \leq 12$ ), exhaustive search gives the guaranteed optimum. For medium instances, NN + 2-opt provides near-optimal solutions quickly. For large real-world instances ( $n > 1000$ ), metaheuristics and Concorde are the tools of choice.

## 1.4 Software Engineering Considerations I

Building a TSP solver involves not just the algorithms but also a careful **software design** that separates concerns and allows different heuristics to be swapped in and out without rewriting the core problem logic.

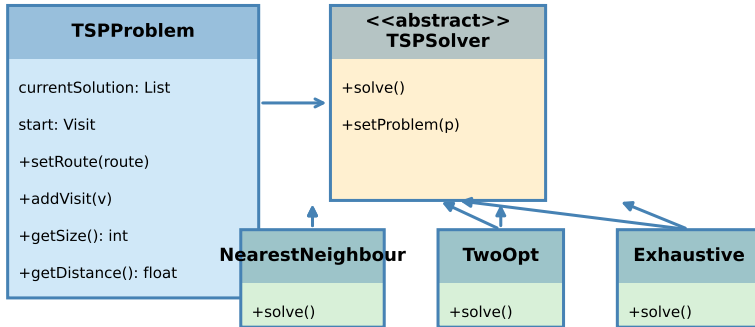
**Representation of a solution.** A TSP solution is naturally represented as an *ordered list* (array or `ArrayList`) of city (`Visit`) objects. This list-of-visits representation supports the two key operations needed:

- 1 **Computing the tour length:** iterate through consecutive pairs and sum the distances.
- 2 **Modifying the tour:** swap, reverse, or insert sub-sequences.

**The Strategy design pattern.** The book advocates keeping the **problem** (data and distance computation) separate from the **heuristic** (the search algorithm). This is an instance of the *Strategy* pattern from object-oriented design. A `TSPSolver` abstract class defines the interface (`solve()`, `setProblem()`), and concrete subclasses (`NearestNeighbour`, `TwoOpt`, `Exhaustive`) each implement the interface differently.

## 1.4 Software Engineering Considerations II

Software design: separating problem from heuristic



**Figure:** The TSP class hierarchy implementing the Strategy pattern. **TSPProblem** holds the data (cities, distance matrix, current solution). **TSPSolver** is an abstract class with a `solve()` method. Each concrete subclass

## 1.4 Software Engineering Considerations III

(`NearestNeighbour`, `TwoOpt`, `Exhaustive`) implements a different algorithm but is interchangeable through the common interface.

## 1.4 Software Engineering Considerations IV

### Key design benefits.

- **Extensibility:** Adding a new algorithm (e.g., 3-opt, a genetic algorithm) requires only adding a new subclass — the rest of the system remains unchanged.
- **Testability:** Each algorithm can be unit-tested independently of the others.
- **Maintainability:** Domain experts writing heuristics do not need to understand the data-loading or distance-computation code, and vice versa.
- **Comparability:** By calling `solver.solve(problem)` with different solver objects, the case study can compare all algorithms on the same problem instance with minimal code change.

**Measuring distance: the Haversine formula.** The case study uses real-world geographic coordinates (latitude/longitude). Straight-line Euclidean distance is inappropriate for a spherical Earth. Instead, the **Haversine formula** computes the great-circle distance between two points:

$$d = 2R \arcsin \left( \sqrt{\sin^2 \left( \frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left( \frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$



## 1.4 Software Engineering Considerations V

### Haversine notation

- $\phi$  (phi): latitude in radians (the angle above the equator; ranges from  $-\pi/2$  to  $+\pi/2$ ).
- $\lambda$  (lambda): longitude in radians (the angle east of the prime meridian; ranges from  $-\pi$  to  $+\pi$ ).
- $R$ : Earth's mean radius  $\approx 6371$  km.
- $\arcsin$ : the inverse sine function, returning an angle.
- The formula gives the shortest path over the surface of a sphere between two points.

### Key takeaway: software design

Separating the problem representation from the heuristic is not merely good style — it makes the software genuinely easier to extend and compare. This pattern will be reused in every chapter of the book.

## 1.5 A TSP Case Study: Santa's Delivery Problem I

**Problem description.** The case study is a playful but technically rich problem: how should **Santa Claus** plan his sleigh route to deliver presents?

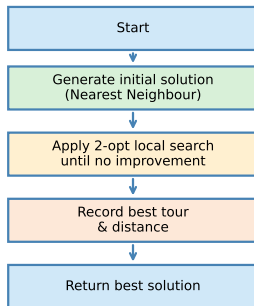
Santa leaves his grotto in Lapland, visits the house of each child (one per country/region), delivers presents, and returns to the grotto. The dataset contains 50 visits (children's locations) distributed across the world, with latitude/longitude coordinates. Santa uses the Haversine distance to measure how far apart any two locations are.

**Goal.** Compare three algorithms — **Exhaustive Search**, **Nearest Neighbour**, and **2-opt** (and their hybrid combination) — on instances of increasing size from 1 to 50 visits, measuring both *solution quality* (total distance in km) and *execution time* (in ms).

## 1.5 A TSP Case Study: Santa's Delivery Problem II

**The Hybrid solver.** The best practical result is achieved by a **Hybrid** algorithm that first runs Nearest Neighbour to generate a reasonable initial tour, then applies 2-opt to improve it:

Hybrid algorithm: NN initialisation + 2-opt refinement



## 1.5 A TSP Case Study: Santa's Delivery Problem III

**Figure:** The Hybrid solver: NN builds an initial tour quickly; 2-opt then iteratively refines it until no 2-edge swap gives further improvement. This combination consistently outperforms either algorithm used alone.

## 1.5 A TSP Case Study: Santa's Delivery Problem IV

**Experimental setup.** Each algorithm is run 10 times on each instance size, and the results are reported as  $\min(\text{average})$  over 10 runs. For deterministic algorithms (Nearest Neighbour from a fixed start), only one run is needed. The Exhaustive Search is excluded from instances with  $n > 13$  cities because its running time becomes prohibitive.

# Python: Exhaustive Search Algorithm

Listing 1: Exhaustive search — Python translation of the book's Java Listing 1.1

```
1 import itertools
2
3 def exhaustive_tsp(problem):
4     """
5     Enumerate every permutation of cities (fixing the start city)
6     and return the shortest complete tour.
7     problem.cities : list of city objects
8     problem.dist(a,b): distance between two cities
9     """
10    cities = problem.cities
11    start = cities[0]
12    rest = cities[1:]          # cities[1:] = all except the fixed start
13
14    best_dist = float('inf')   # infinity will be beaten by any real tour
15    best_route = None
16
17    # itertools.permutations generates all (n-1)! orderings of rest
18    for perm in itertools.permutations(rest):
19        route = [start] + list(perm) + [start] # complete circuit
20        dist = sum(problem.dist(route[i], route[i+1])
21                    for i in range(len(route)-1))
22        if dist < best_dist:      # found a shorter tour
23            best_dist = dist
24            best_route = route
```

# Python: Nearest Neighbour Algorithm

## Listing 2: Nearest Neighbour heuristic — Python translation of book Listing 1.2

```
1 def nearest_neighbour_tsp(problem):
2     """
3     Build a tour greedily: always go to the nearest unvisited city.
4     Returns: (tour as list of cities, total distance)
5     """
6     cities      = problem.cities[:]      # copy so we can modify
7     start       = problem.start          # fixed starting city
8     current     = start
9     route       = [current]
10    not_visited  = [c for c in cities if c != start]
11
12    while not_visited:                    # while there are unvisited cities
13        best_d = float('inf')
14        next_c = None
15        for possible in not_visited:
16            d = problem.dist(current, possible)
17            if d < best_d:
18                best_d = d
19                next_c = possible          # closest unvisited city found so far
20        not_visited.remove(next_c)        # mark as visited
21        route.append(next_c)              # add to tour
22        current = next_c                  # move there
23
24    route.append(start)                   # return to start
25    total = sum(problem.dist(route[i], route[i+1])
```

# Python: 2-opt Algorithm

Listing 3: 2-opt heuristic — Python translation of book Listing 1.3

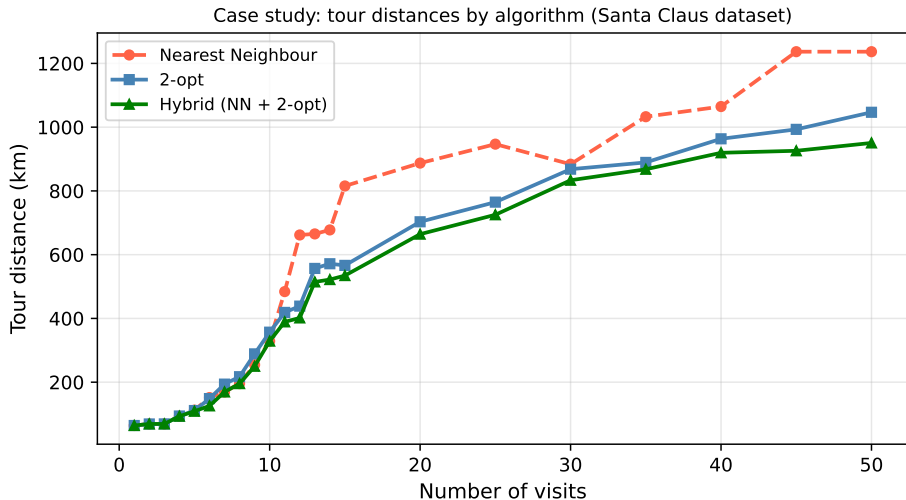
```
1 def two_opt_swap(route, i, j):
2     """
3     Reverse the sub-sequence between positions i and j (inclusive).
4     route: list of city objects (does NOT include return-to-start)
5     """
6     return route[:i] + route[i:j+1][::-1] + route[j+1:]
7
8 def two_opt_tsp(problem):
9     """
10    Improve an initial (e.g. NN) tour by repeatedly applying 2-opt swaps.
11    """
12    route = nearest_neighbour_tsp(problem)[0][:-1] # remove trailing start
13    improved = True
14
15    while improved: # keep scanning until no swap helps
16        improved = False
17        best_dist = problem.get_distance(route)
18
19        for i in range(len(route)):
20            for j in range(i+1, len(route)):
21                new_route = two_opt_swap(route, i, j)
22                new_dist = problem.get_distance(new_route)
23                if new_dist < best_dist: # improvement found
24                    route = new_route
25                    best_dist = new_dist
```



## 1.5 Results and Analysis I

**Tour quality comparison.** The figure below shows the best tour distance found by each algorithm as the number of visits increases from 1 to 50.

## 1.5 Results and Analysis II



**Figure:** Tour distance vs. instance size for the three algorithms on the Santa Claus dataset. The Hybrid solver (NN + 2-opt) gives the shortest tours for all instance sizes above 10 visits. For small instances ( $n < 13$ ) the Exhaustive

## 1.6 Conclusions I

### What we have learned in Chapter 1.

**The TSP is the canonical hard delivery problem.** Any problem that requires visiting a set of locations optimally is, at its core, a TSP or a variant of it. Understanding the TSP — its complexity, its solution methods, and its software design — provides the foundation for every subsequent chapter.

**The combinatorial explosion is the fundamental difficulty.** The number of possible routes grows as  $(n - 1)!/2$ . For  $n = 20$  cities this is already  $6 \times 10^{16}$ , making exhaustive search completely impractical. This is why heuristics are not a compromise but a *necessity*.

**NP-hardness explains why no “perfect” polynomial-time algorithm exists.** Unless  $P = NP$  (which is widely believed to be false), the TSP cannot be solved exactly in polynomial time. The best exact algorithms (Branch and Bound, Concorde) use clever bounding and cutting-plane techniques but still require exponential time in the worst case.

**Heuristics provide practical solutions in reasonable time.** Nearest Neighbour runs in  $O(n^2)$  and gives tours roughly 20–25% above optimal. 2-opt (also  $O(n^2)$  per pass) systematically removes crossing edges and reduces tour length by a further 10–20%. Combining them (Hybrid: NN + 2-opt) consistently gives the best practical results and runs in milliseconds for 50 cities.

## 1.6 Conclusions II

**Software design matters as much as the algorithm.** Separating the *problem* (the `TSPProblem` class with city data and distance computation) from the *solver* (the abstract `TSPSolver` class and its concrete subclasses) follows the Strategy pattern. This design makes it easy to add new algorithms, compare them fairly, and maintain the codebase as the project grows.

**Looking ahead.** The methods studied in Chapter 1 are foundational, but they have limitations: 2-opt gets stuck in local optima, and the Hybrid has no mechanism to escape them. The remaining chapters introduce *nature-inspired metaheuristics* — Evolutionary Algorithms, Ant Colony Optimisation, and others — that use population-based or stochastic acceptance strategies to explore the solution space more broadly and find higher-quality solutions on large, real-world delivery instances.

### Chapter summary in one sentence

The TSP captures the core difficulty of delivery optimisation — NP-hard combinatorial search — and the Nearest Neighbour plus 2-opt heuristics provide a fast, practical baseline that later nature-inspired methods aim to surpass.

# References I

- **Applegate, D., Bixby, R., Chvátal, V., Cook, W.** (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.
- **Bocks** (1958). Solving the TSP on an IBM 650 computer.
- **Croes, G.A.** (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791–812.
- **Flood, M.M.** (1956). The traveling-salesman problem. *Operations Research*, 4(1), 61–75.
- **Hamilton, W.R.** (1857). The Icosian Game. Exhibited at the British Association.
- **Lin, S., Kernighan, B.W.** (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2), 498–516.
- **OpenStreetMap contributors** (2021). OpenStreetMap.  
<https://www.openstreetmap.org/copyright>
- **Springer** (2022). *Nature Inspired Optimisation for Delivery Problems*. Studies in Computational Intelligence.